JobATS

INTRODUCTION:

JobATS (Job Application Tracking System) is a full-stack web application developed using the MERN stack (MongoDB, Express.js, React.js, and Node.js). The system is designed to simplify and automate the recruitment process for both candidates and recruiters .In traditional hiring systems, managing job applications manually through emails and spreadsheets becomes inefficient and time-consuming. JobATS provides a centralized platform where recruiters can post jobs, review applications, and manage hiring workflows, while candidates can explore opportunities, apply for jobs, and track their application status. The system enhances transparency, improves communication, and ensures a smooth hiring experience.

SCENARIO Based Case Study:

Meet Yoshitha, a job seeker with a busy academic schedule who values organization and efficiency in her job search process. Yoshitha is actively looking for job opportunities such as Data Analyst and Full Stack Developer roles, which requires her to stay updated on available jobs, manage applications, and track her progress throughout the recruitment process.

Yoshitha discovers JobATS, a comprehensive web application designed to streamline job searching and recruitment operations. Intrigued by its features, she decides to explore how JobATS can help her improve her productivity and manage her job applications effectively.

User Registration and Authentication: Yoshitha registers an account on JobATS by providing her personal details and selecting her role as a candidate. She logs in securely using her credentials, ensuring that her data remains private and protected.

Job Browsing and Search: Yoshitha explores JobATS's job listings, which include opportunities such as Data Analyst at Oracle and Full Stack Developer at Deloitte. Each job provides detailed information like location, salary, and job description. She can easily search and filter jobs to find relevant opportunities.

Application Management: Yoshitha applies for jobs directly through JobATS. Once applied, the system records her application and allows her to track its status. She can view whether her application is in Applied, Interview, Selected, or Rejected stage, helping her stay organized.

Dashboard Overview: JobATS provides Yoshitha with a dashboard that displays key information such as total jobs available, number of applications, interview status, and selection progress. This helps her monitor her job search activity in one place.

Recruiter Management: On the recruiter side, Koushika uses JobATS to manage job postings and applicants. She can create new jobs, edit or delete existing listings, and review candidates who have applied. She can also update the application status of candidates, ensuring smooth communication in the hiring process.

Application Tracking System (ATS): JobATS includes an Application Tracking System that organizes all applications into different stages such as Applied, Interview, Selected, and Rejected. This structured pipeline helps both candidates and recruiters track progress efficiently.

User Interface and Experience: JobATS provides a clean and user-friendly interface with clear navigation between Dashboard, Jobs, and Applications. The system is designed to make job searching and recruitment simple and efficient for all users.

Yoshitha's Experience: Thanks to JobATS, Yoshitha can now manage her job search more efficiently and effectively. She can easily browse jobs, apply for positions, and track her progress without confusion. JobATS has become a valuable tool in her career journey, helping her stay organized and focused on achieving her professional goals.

SYSTEM REQUIREMENTS:

Software Requirements:

To ensure smooth development, deployment, and usage of the **JobATS Web-Application**, certain system prerequisites must be met. These requirements are categorized into software, setup, and hardware specifications, all of which contribute to building a robust and scalable job application management platform.

1. Software Requirements

These are the essential tools and platforms required to develop, test, and run the application efficiently.

- **Operating System:** Windows 10/11, macOS, or Linux — Supports cross-platform development and testing.
- **Node.js (v16 or above):** Provides the runtime environment for building and managing frontend logic and powers the server-side logic while handling API

routing.

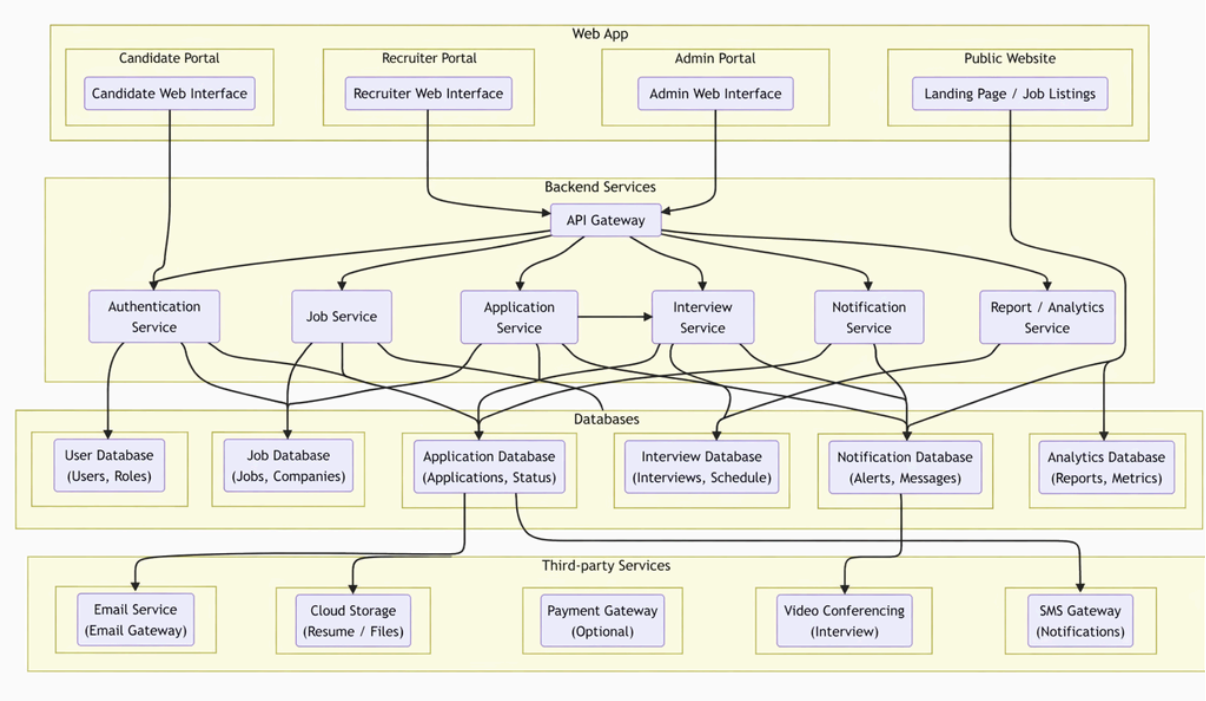
- **npm (v8 or above):** A package manager required to install dependencies for React and other libraries used in the application.
- **React.js:** A JavaScript library used for building dynamic and responsive user interfaces for the JobATS platform.
- **Browser:** Google Chrome / Firefox (latest version) — Used for rendering and testing the user interface in real time.
- **Express.js:** A lightweight web framework used for building RESTful APIs and handling backend operations.
- **MongoDB:** A NoSQL database used to store structured and unstructured data such as users, jobs, applications, and interview details.
- **Postman:** Tool used for testing API endpoints during backend development and debugging.
- **Visual Studio Code:** Preferred code editor with built-in Git support and terminal integration for efficient development.

2. Hardware Requirements

- Describes the minimum and recommended specifications needed to support the development and usage of the application.
- **Processor:** Intel Core i5 (8th Gen or above) / AMD Ryzen 5 or better — Ensures smooth performance while running development tools and servers simultaneously.
- **RAM:** Minimum 8 GB (16 GB recommended) — Required for handling multiple processes such as frontend server, backend server, and database operations efficiently.
- **Storage:** At least 1 GB free space — Required for installing dependencies, storing project files, and maintaining the database.
- **Display:** 1366×768 resolution or higher — Recommended for better visualization of the application interface and development environment.

PROJECT ARCHITECTURE:

TECHNICAL ARCHITECTURE:



In this architecture, the flow begins with a user (represented by the "User" node) interacting with the JobATS frontend (represented by the "JobATS Frontend" node). The user submits actions such as registering, logging in, browsing jobs, or applying for a job through the frontend interface.

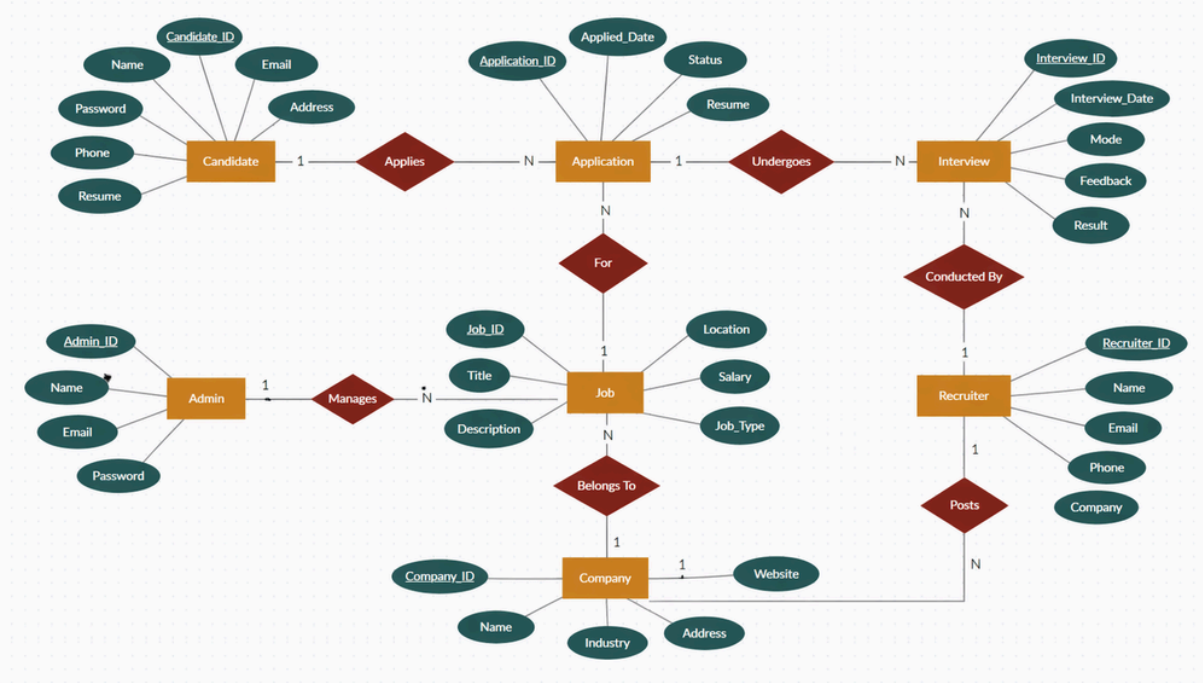
The JobATS frontend then sends a request to the JobATS backend (represented by the "JobATS Backend" node). The backend processes the request and handles the corresponding operation such as fetching job listings, storing application details, or updating application status.

The backend communicates with a database (represented by the "Database" node) to store and retrieve data securely. The database stores the application data, including user details, job information, and application status such as Applied, Interview, Selected, and Rejected.

Additionally, the backend may utilize a file storage system (represented by the "File Storage" node) to store any files or attachments associated with job applications. This can include uploaded resumes or other relevant documents submitted by candidates.

Once the backend has processed the request, it sends a response back to the JobATS frontend. The frontend then displays the updated information to the user, such as job details, application status, or confirmation messages.

ER DIAGRAM:



The Entity-Relationship (ER) diagram for JobATS represents the various entities and their relationships within the job application tracking system. Here is a description of the key components of the ER diagram:

1. Entities:

- **Candidate:** Represents users who apply for jobs in the system, with attributes such as candidate ID, name, email, phone, password, and resume.
- **Job:** Represents job listings available in the system, with attributes such as job ID, title, description, location, salary, and job type.
- **Application:** Represents job applications submitted by candidates, with attributes such as application ID, applied date, status, and resume.
- **Recruiter:** Represents recruiters who post and manage job listings, with attributes such as recruiter ID, name, email, phone, and company.
- **Company:** Represents organizations that offer jobs, with attributes such as company ID, name, industry, address, and website.
- **Interview:** Represents interview processes for candidates, with attributes such as interview ID, interview date, mode, feedback, and result.
- **Admin:** Represents system administrators who manage the platform, with attributes such as admin ID, name, email, and password.

2. Relationships:

- **Candidate Application:** Represents the relationship between candidates and applications, indicating which candidate applies for which job.
- **Application Job:** Represents the relationship between applications and jobs,

indicating which job a particular application is for.

- **Recruiter Job:** Represents the relationship between recruiters and jobs, indicating which recruiter posts which job.
- **Job Company:** Represents the relationship between jobs and companies, indicating which company offers each job.
- **Application Interview:** Represents the relationship between applications and interviews, indicating the interview process for each application.
- **Recruiter Interview:** Represents the relationship between recruiters and interviews, indicating which recruiter conducts the interview.
- **Admin Job:** Represents the relationship between admin and jobs, indicating that admin manages job listings in the system.

3. Attributes:

- Each entity has its own set of attributes that describe its properties. For example, the Candidate entity has attributes such as candidate ID, name, email, phone, and resume.

4. Primary Keys:

- Each entity has a primary key that uniquely identifies each record in the entity. For example, the Candidate entity's primary key is the candidate ID.

5. Foreign Keys:

- Foreign keys are used to establish relationships between entities. For example, the Application entity may contain foreign keys such as candidate ID and job ID to link applications with candidates and jobs.

Overall, the ER diagram for JobATS provides a visual representation of the database schema, showing how different entities are related and how data flows between them in the job application tracking system.

Key Features:

The key features of the JobATS application can vary depending on its functionality and user roles. However, here are the common key features included in the JobATS platform.

User Registration and Authentication:
Allows users (candidates and recruiters) to create accounts, log in securely, and

manage their personal information while ensuring data privacy.

Job Management:
Enables recruiters to create, update, and delete job postings, including details such as job title, description, location, and salary.

Job Search and Application:
Allows candidates to browse available jobs, filter based on preferences, and apply directly through the platform.

Application Tracking System:
Helps track the status of applications (Applied, Interview, Selected, Rejected), allowing both candidates and recruiters to monitor progress efficiently.

Dashboard and Analytics:
Provides users with a dashboard overview showing key metrics such as number of jobs, applications, and interview status for better decision-making.

Resume Management:
Allows candidates to upload and manage resumes while enabling recruiters to view and evaluate candidate profiles.

Interview Management:
Enables recruiters to schedule interviews and update candidate status based on interview outcomes.

Role-Based Access:
Supports different user roles (Candidate and Recruiter) with customized functionalities and access permissions.

ROLES AND RESPONSIBILITIES:

Candidate (User):

- **Registration and Account Management:** Candidates are responsible for creating an account, providing accurate personal information, and managing their profile details, including updating information and maintaining password security.
- **Job Search and Application:** Candidates can search for available job opportunities based on preferences such as role, location, and salary. They are responsible for selecting suitable jobs and applying with the required details and resume.
- **Application Tracking:** Candidates can monitor the status of their job

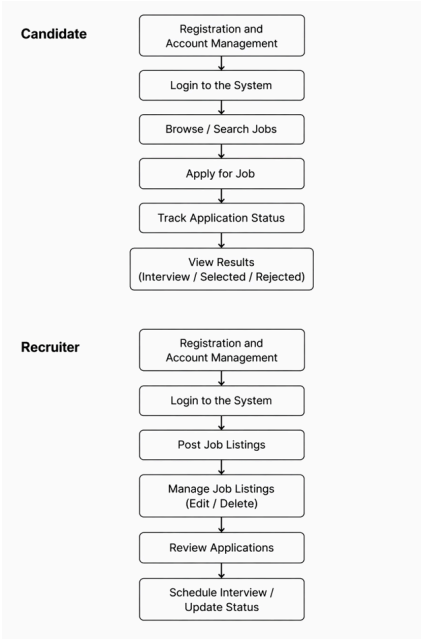
applications (Applied, Interview, Selected, Rejected) and stay updated on their progress throughout the hiring process.

- **Resume Management:** Candidates are responsible for uploading and maintaining updated resumes to improve their chances of selection.
- **Communication:** Candidates may need to communicate with recruiters regarding job applications, interviews, or updates, and should use the platform responsibly.

Recruiter:

- **Registration and Account Management:** Recruiters are responsible for creating and managing their accounts, ensuring that company and contact details are accurate and up to date.
- **Job Posting and Management:** Recruiters can create, update, and delete job listings. They are responsible for providing clear and accurate job descriptions, requirements, and other relevant details.
- **Application Review:** Recruiters can view applications submitted by candidates and evaluate them based on qualifications and requirements.
- **Interview Management:** Recruiters are responsible for scheduling interviews, updating candidate status, and managing the hiring pipeline.
- **Candidate Communication:** Recruiters may communicate with candidates for interview scheduling, feedback, or selection updates.

User Flow:



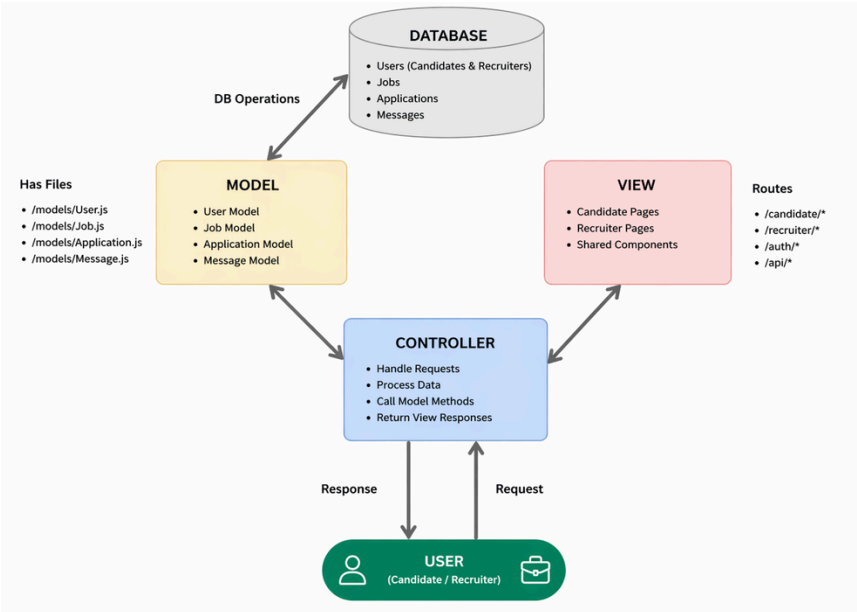
Candidate Flow

- **Registration & Account –** Create and manage account details securely.
- **Login –** Access the system using registered credentials.
- **Browse Jobs –** Search and view available job listings based on preferences.
- **Apply for Jobs –** Submit applications with required details and resume.
- **Application Tracking –** Monitor application status (Applied, Interview, Selected, Rejected).
- **Communication –** Interact with recruiters for updates or interview scheduling.
- **Results –** View final outcome of job applications.

Recruiter Flow

- **Registration & Account –** Create and manage recruiter account details securely.
- **Login –** Access the system using registered credentials.
- **Job Posting –** Create and publish job listings with required details.
- **Job Management –** Edit, update, or delete job postings as needed.
- **Application Review –** View and evaluate candidate applications.
- **Candidate Management –** Update application status (Interview, Selected, Rejected).
- **Communication –** Interact with candidates for interview scheduling and updates.

MVC PATTERN:



The JobATS application follows the Model-View-Controller (MVC) architectural pattern, a software design approach that separates an application into three interconnected layers. This separation allows for modularity, easier maintenance, and scalability.

Model Layer (Data Layer)

The Model layer is responsible for handling all data-related logic. This includes defining data schemas and performing database operations. In JobATS, models are implemented using Mongoose, which provides a schema-based solution to interact with MongoDB.

The key models include Candidate, Recruiter, Job, and Application, which store and manage all essential data within the system.

View Layer (Frontend/UI Layer)

The View layer is responsible for presenting data to the user through the user interface. In JobATS, the frontend is developed using React.js, which enables dynamic and responsive UI components.

This layer includes candidate dashboards, recruiter dashboards, job listings, and application tracking interfaces.

Controller Layer (Logic Layer)

The Controller layer acts as an intermediary between the Model and View layers. It handles user requests, processes input data, interacts with the Model, and returns the appropriate response to the View.

In JobATS, controllers manage functionalities such as user authentication, job posting, job applications, and status updates.

Workflow of MVC in JobATS

When a user (candidate or recruiter) interacts with the application, the request is sent to the Controller. The Controller processes the request and communicates with the Model to retrieve or update data in the database. The processed data is then sent to the View, which displays the information to the user.

This structure ensures a clean separation of concerns, making the JobATS application more efficient, scalable, and easier to maintain.

Advantages of Using MVC in This Project

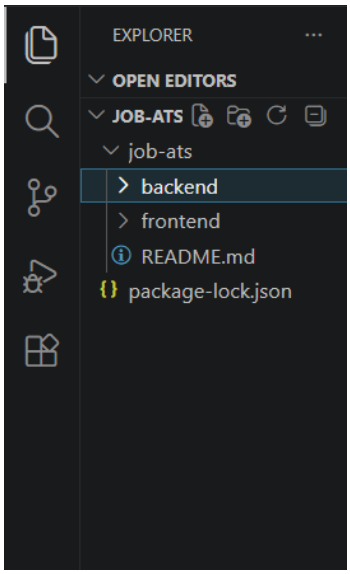
- **Separation of Concerns:** Each layer (Model, View, Controller) has a clearly defined responsibility, improving code readability and maintainability.
- **Scalability:** New features such as job filters, notifications, or interview modules can be added easily by extending models, controllers, and routes.
- **Reusability:** Business logic implemented in controllers and models can be reused across different parts of the application, such as candidate and recruiter functionalities.
- **Testing:** Each layer can be tested independently, making it easier to debug and ensure reliability of core functionalities like authentication and job applications.
- **Collaboration-Friendly:** Multiple developers can work simultaneously on frontend (View) and backend (Model & Controller) without conflicts, improving development efficiency.

Project Setup and Configuration

● Creating Project Folder

1. Create a new folder named **JobATS**.
2. Inside that folder, create two subfolders.
3. Name one folder as **frontend** (Client-side).
4. Name the other folder as **backend** (Server-side).
5. Open the main **JobATS** folder in Visual Studio Code.

PROJECT SETUP AND CONFIGURATION



Client setup (installing React app)

- Open the frontend folder in the terminal of VS Code
  - npm create vite@latest

- Select React framework from the given options
  - React

- Select JavaScript variant
  - JavaScript

- Navigate to frontend folder
  - cd frontend

- Install dependencies
  - npm install

- Run the application
  - npm run dev

Server setup (npm init)

- Open backend folder in terminal
  - npm init -y

- Install dependencies
  - npm install express mongoose cors dotenv

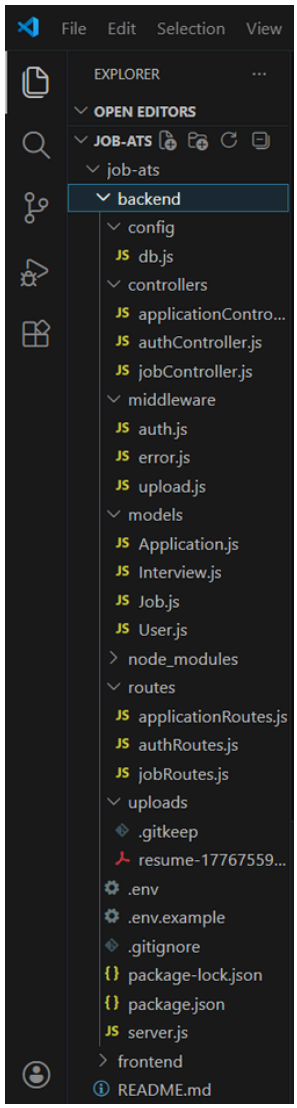
- Create file
  - server.js

- Create folders
  - models
  - controllers
  - routes
  - config

BACKEND DEVELOPMENT:

Backend Reference Video:
https://drive.google.com/file/d/1nCwK0vR060X3kFwljQpFCs0LR_DEcqF-/view?usp=sharing

Backend Structure:



Controllers

● **authController.js** → Handles user authentication (register, login) for candidates and recruiters.

● **jobController.js** → Handles job-related operations (create, update, delete, fetch jobs).

● applicationController.js → Handles job application logic (apply, update status, fetch applications).config

● **db.js** → Contains database connection setup (MongoDB connection using Mongoose).

middleware

● **authMiddleware.js** → Middleware for authentication and authorization (JWT validation).

models

- **User.js** → Schema/model for users (candidate and recruiter details).
- **Job.js** → Schema/model for job postings.
- **Application.js** → Schema/model for storing job applications and their status.

routes

- **authRoutes.js** → Defines API routes for authentication (register, login).
- **jobRoutes.js** → Defines API routes for job operations.
- **applicationRoutes.js** → Defines API routes for job applications and status updates.

DATABASE DEVELOPMENT:

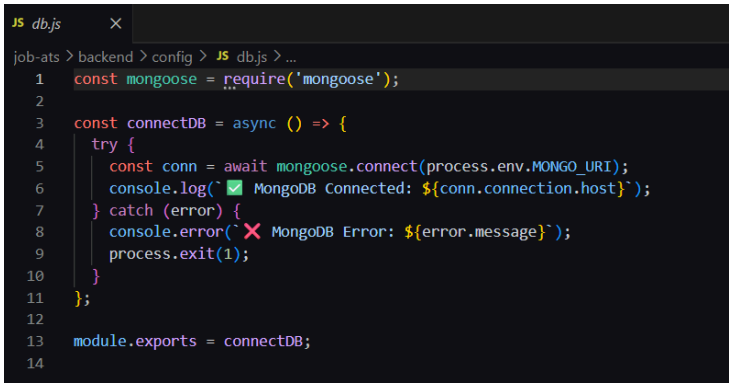
1. Configure MongoDB:

- Install Mongoose.
  In your Node.js project directory, run:
  npm install mongoose

- Create database connection.

- Create Schemas & Models.

Create Database connection:
Now, make sure the database is connected before performing any of the actions through the backend. The connection code looks similar to the one provided below, where MongoDB is connected using Mongoose to store and manage data such as users, jobs, and applications.

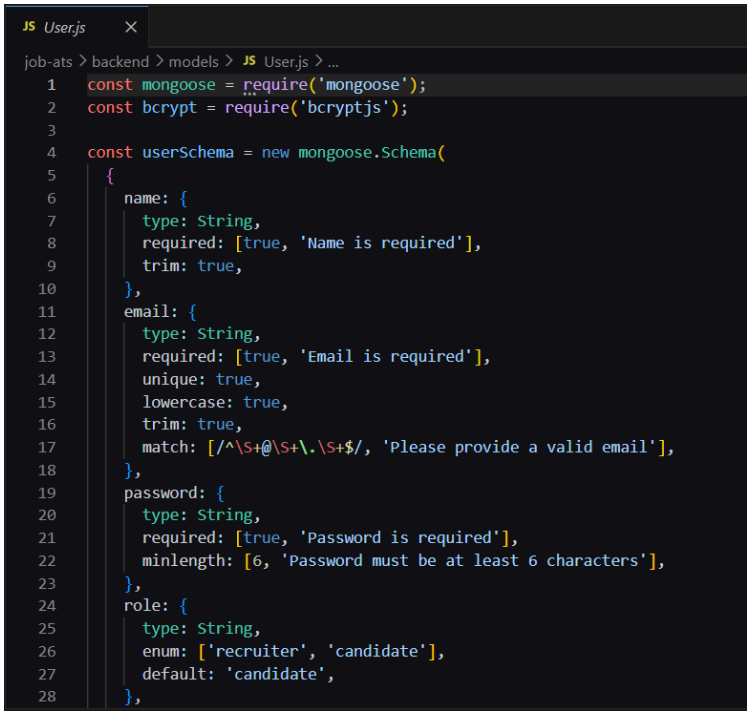


Create Schemas:

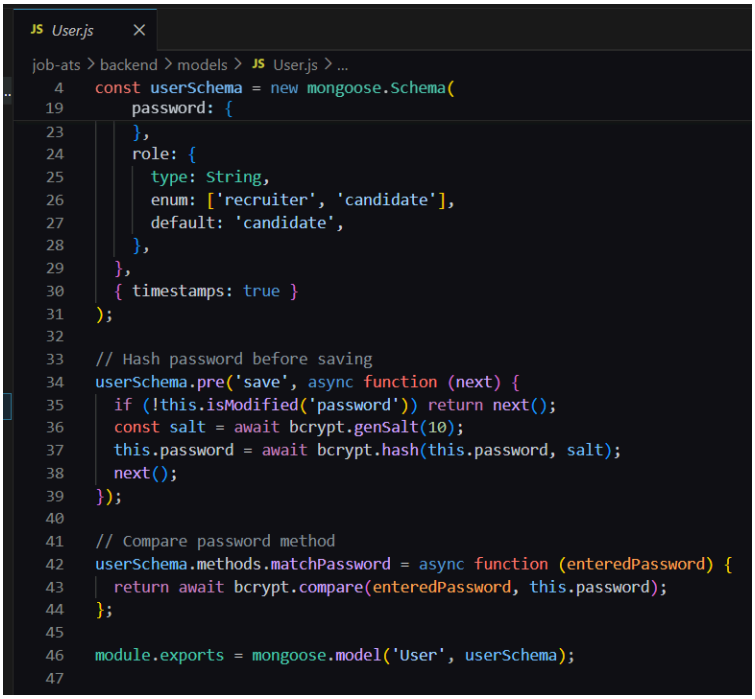
Firstly, configure the Schemas for MongoDB database, to store the data in such a

pattern. Use the data from the ER diagrams to create the schemas. The Schemas for this application look alike to the ones provided below, where models such as User, Job, and Application are defined using Mongoose to structure and manage the data efficiently.

User.js → Schema/model for user accounts.



```
JS User.js
job-ats > backend > models > JS User.js > ...
1  const mongoose = require('mongoose');
2  const bcrypt = require('bcryptjs');
3
4  const userSchema = new mongoose.Schema(
5    {
6      name: {
7        type: String,
8        required: [true, 'Name is required'],
9        trim: true,
10     },
11     email: {
12       type: String,
13       required: [true, 'Email is required'],
14       unique: true,
15       lowercase: true,
16       trim: true,
17       match: [/^\S+@\S+\.\S+$/, 'Please provide a valid email'],
18     },
19     password: {
20       type: String,
21       required: [true, 'Password is required'],
22       minlength: [6, 'Password must be at least 6 characters'],
23     },
24     role: {
25       type: String,
26       enum: ['recruiter', 'candidate'],
27       default: 'candidate',
28     },
```



```
JS User.js
job-ats > backend > models > JS User.js > ...
4  const userSchema = new mongoose.Schema(
19     password: {
23     },
24     role: {
25       type: String,
26       enum: ['recruiter', 'candidate'],
27       default: 'candidate',
28     },
29   },
30   { timestamps: true }
31 );
32
33 // Hash password before saving
34 userSchema.pre('save', async function (next) {
35   if (!this.isModified('password')) return next();
36   const salt = await bcrypt.genSalt(10);
37   this.password = await bcrypt.hash(this.password, salt);
38   next();
39 });
40
41 // Compare password method
42 userSchema.methods.matchPassword = async function (enteredPassword) {
43   return await bcrypt.compare(enteredPassword, this.password);
44 };
45
46 module.exports = mongoose.model('User', userSchema);
47
```

Job.js → Schema/model for storing job listings.

Job.js

job-ats > backend > models > Job.js > ...

```
1   const mongoose = require('mongoose');
2
3   const jobSchema = new mongoose.Schema(
4     {
5       title: {
6         type: String,
7         required: [true, 'Job title is required'],
8         trim: true,
9       },
10      description: {
11        type: String,
12        required: [true, 'Job description is required'],
13      },
14      company: {
15        type: String,
16        required: [true, 'Company is required'],
17        trim: true,
18      },
19      location: {
20        type: String,
21        required: [true, 'Location is required'],
22        trim: true,
23      },
24      salary: {
25        type: String,
26        default: 'Not disclosed',
27      },
28      createdBy: {
```

Job.js

job-ats > backend > models > Job.js > ...

```
3   const jobSchema = new mongoose.Schema(
10      description: {
13      },
14      company: {
15        type: String,
16        required: [true, 'Company is required'],
17        trim: true,
18      },
19      location: {
20        type: String,
21        required: [true, 'Location is required'],
22        trim: true,
23      },
24      salary: {
25        type: String,
26        default: 'Not disclosed',
27      },
28      createdBy: {
29        type: mongoose.Schema.Types.ObjectId,
30        ref: 'User',
31        required: true,
32      },
33    },
34    { timestamps: true }
35  );
36
37  module.exports = mongoose.model('Job', jobSchema);
38
```

Application.js → Schema/model for storing job applications and their status.

Application.js

job-ats > backend > models > Application.js > ...

```
const mongoose = require('mongoose');

const applicationSchema = new mongoose.Schema(
  {
    userId: {
      type: mongoose.Schema.Types.ObjectId,
      ref: 'User',
      required: true,
    },
    jobId: {
      type: mongoose.Schema.Types.ObjectId,
      ref: 'Job',
      required: true,
    },
    resume: {
      type: String,
      required: [true, 'Resume is required'],
    },
    status: {
      type: String,
      enum: ['Applied', 'Interview', 'Selected', 'Rejected'],
      default: 'Applied',
    },
  },
  { timestamps: true }
);

```

Application.js

job-ats > backend > models > Application.js > ...

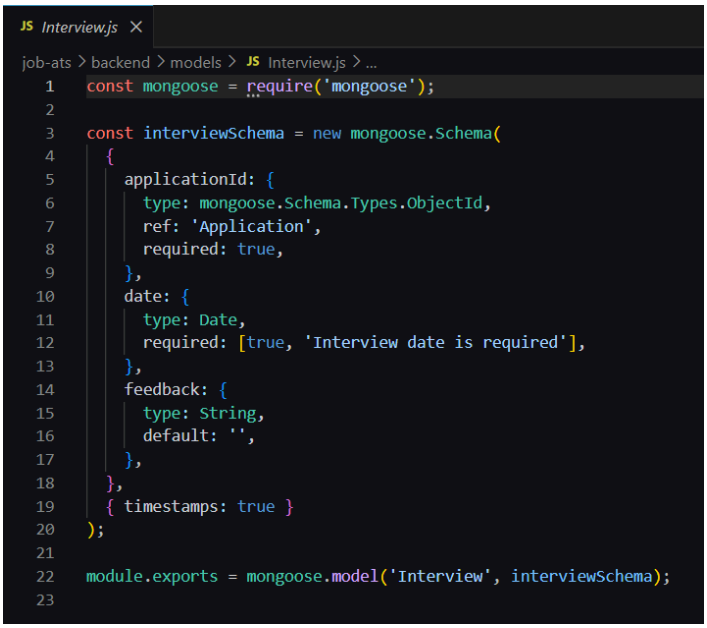
```
const applicationSchema = new mongoose.Schema(
    userId: {
      ref: 'User',
      required: true,
    },
    jobId: {
      type: mongoose.Schema.Types.ObjectId,
      ref: 'Job',
      required: true,
    },
    resume: {
      type: String,
      required: [true, 'Resume is required'],
    },
    status: {
      type: String,
      enum: ['Applied', 'Interview', 'Selected', 'Rejected'],
      default: 'Applied',
    },
  },
  { timestamps: true }
);

// Prevent duplicate applications
applicationSchema.index({ userId: 1, jobId: 1 }, { unique: true });

module.exports = mongoose.model('Application', applicationSchema);

```

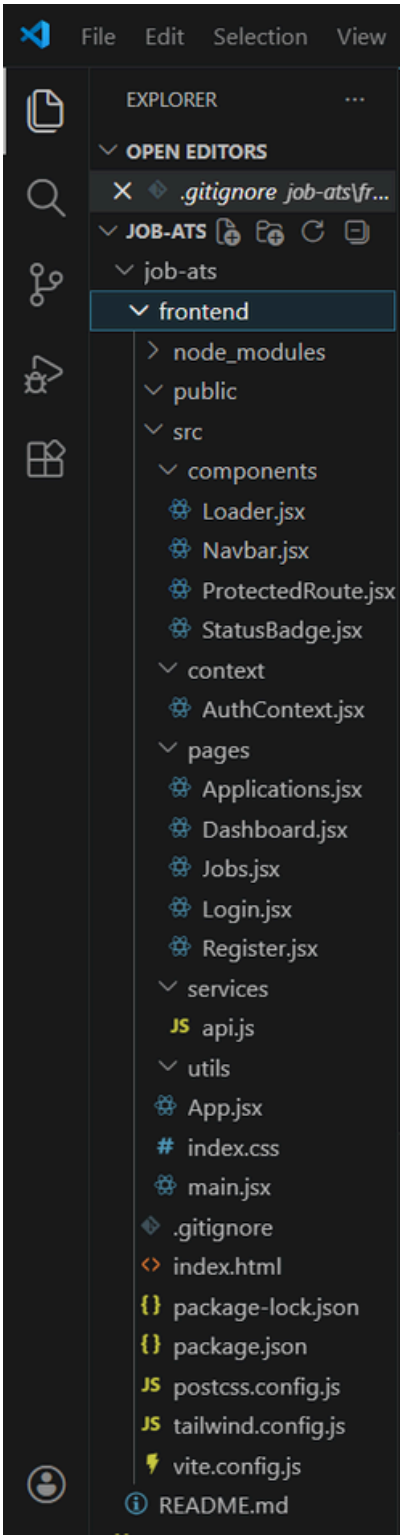
Interview.js → Schema/model for storing interview details and feedback.



FRONT-END DEVELOPMENT:

Frontend Reference Video:
https://drive.google.com/file/d/1RAva4jR9VrZXJwayZ_CpFyIGgHixFd3f/view?usp=sharing

FRONT END STRUCTURE



Frontend (src)

- Login.jsx → Login page for users (candidate/recruiter authentication).
- Register.jsx → Signup page for new users with role selection.
- Dashboard.jsx → Displays user dashboard with overview and stats.

- Jobs.jsx → Shows available job listings and allows candidates to apply.
- Applications.jsx → Displays applied jobs and their current status.
- Navbar.jsx → Navigation bar for routing between pages.

Candidate (src/Candidate)

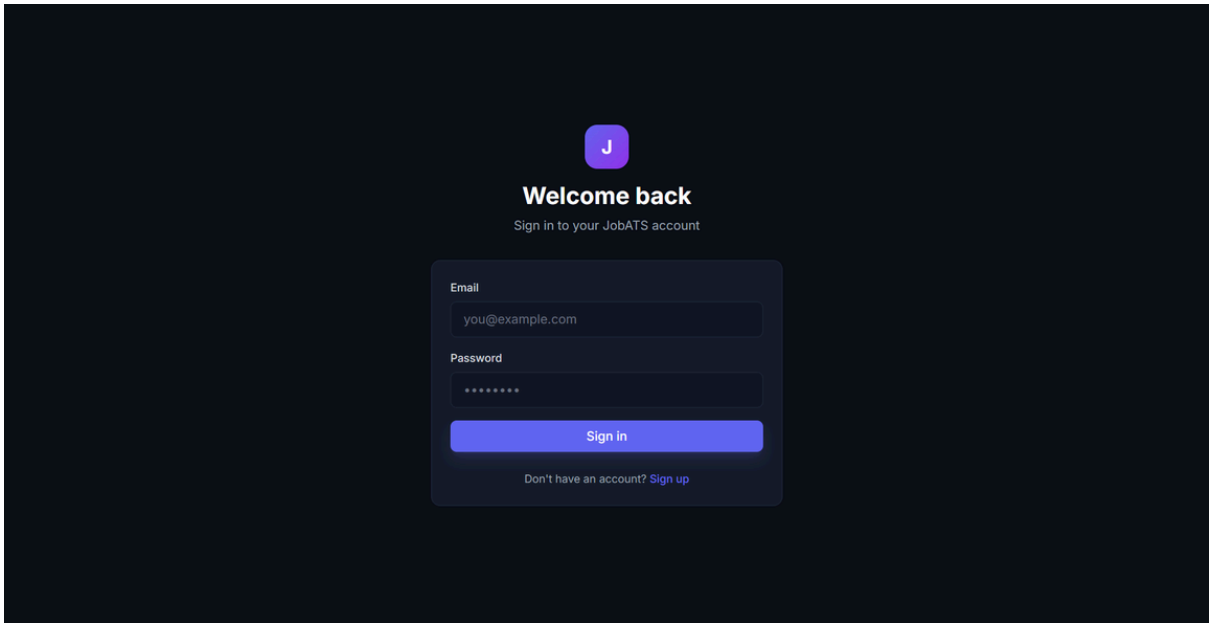
- Dashboard.jsx → Candidate dashboard showing overview and application stats.
- Jobs.jsx → Displays available job listings for candidates to browse and apply.
- Applications.jsx → Shows all applied jobs with their current status.
- Login.jsx → Candidate login page.
- Register.jsx → Candidate signup/registration page.
- Navbar.jsx → Navigation bar for candidate dashboard and pages.

Components (Reusable parts)

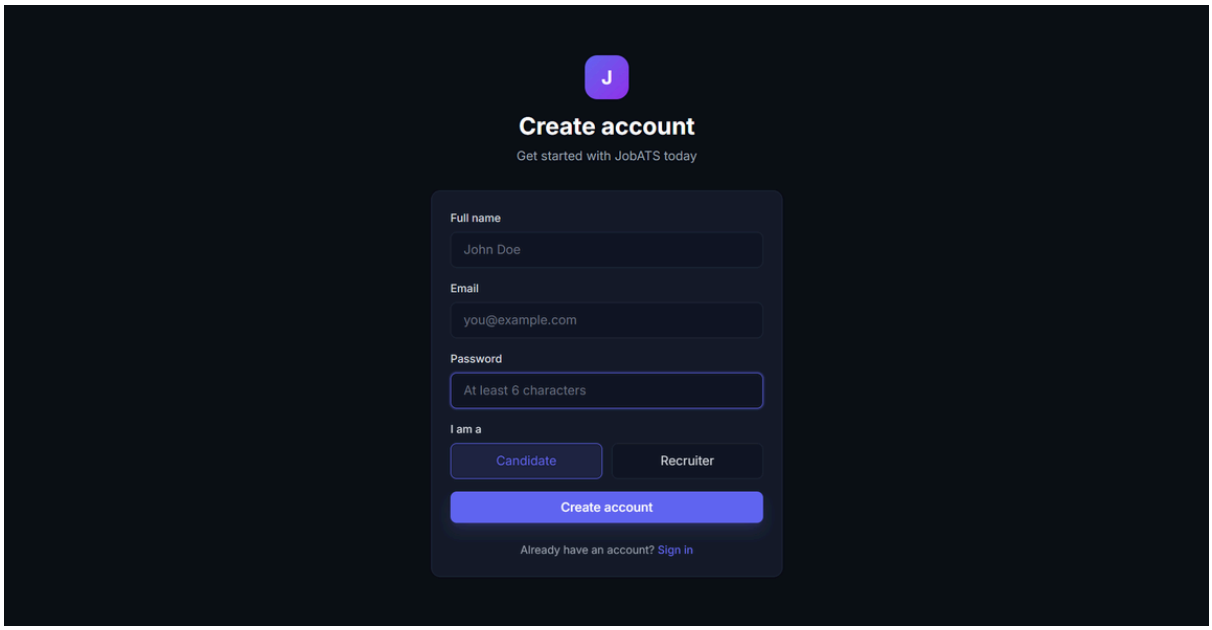
- Navbar.jsx → Common navigation bar used across different pages.
- Home.jsx → Landing page displaying introduction and main features of the application.
- ui/ → Contains reusable UI components such as buttons, cards, and other common elements.

Output ScreenShots:

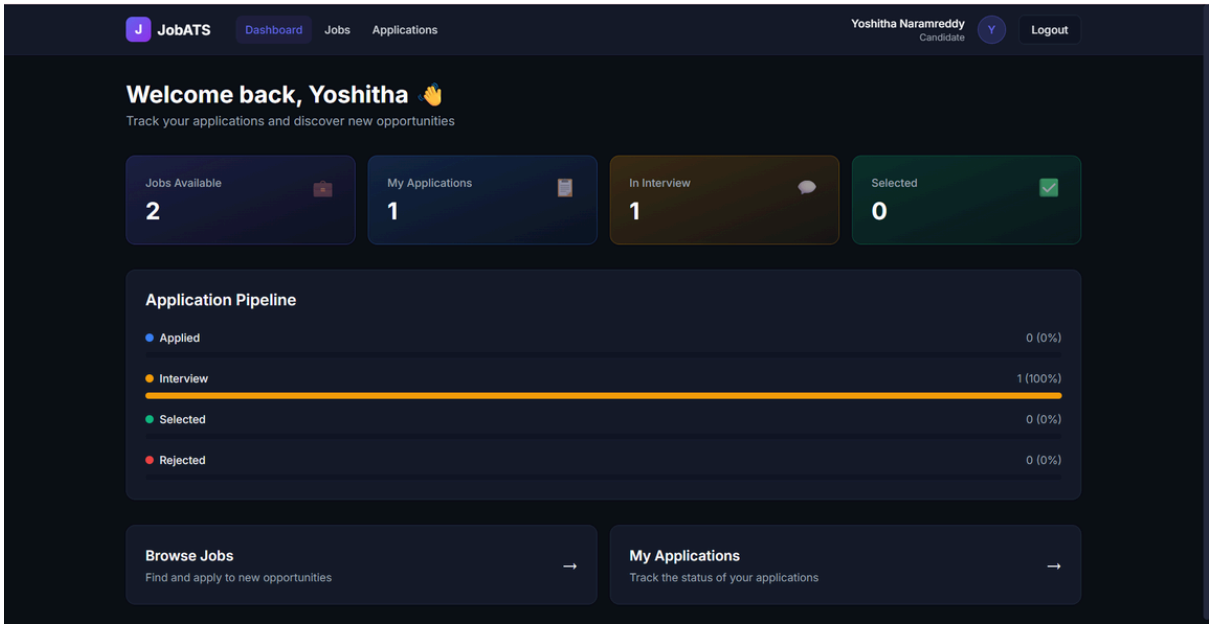
Landing Page:



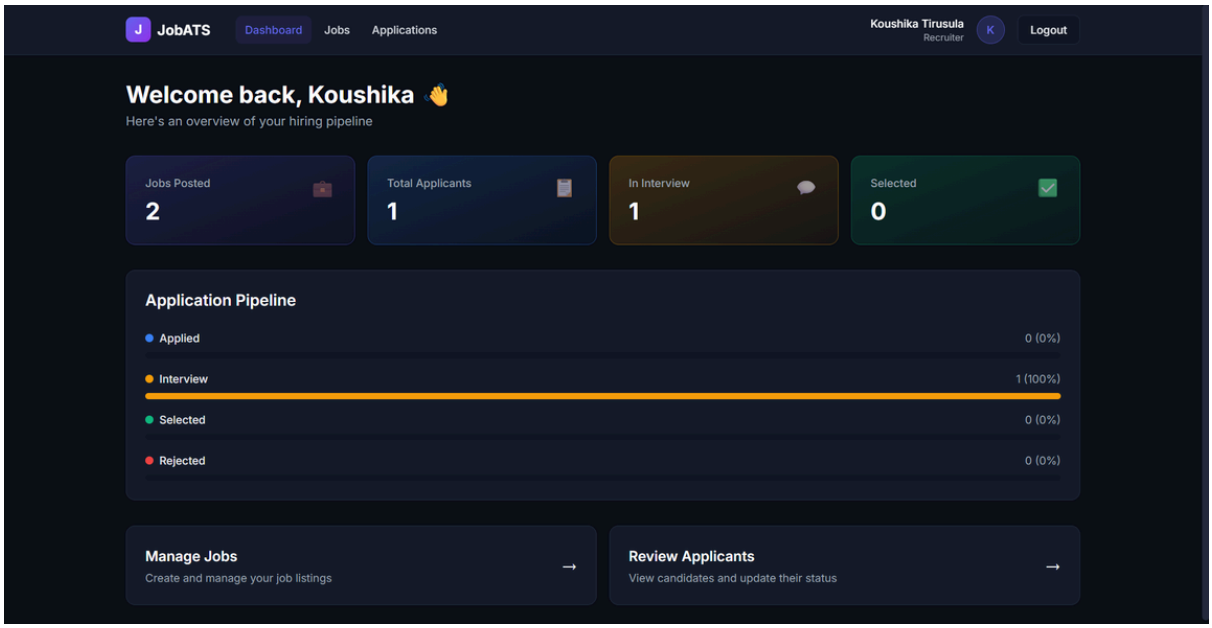
Register Page:



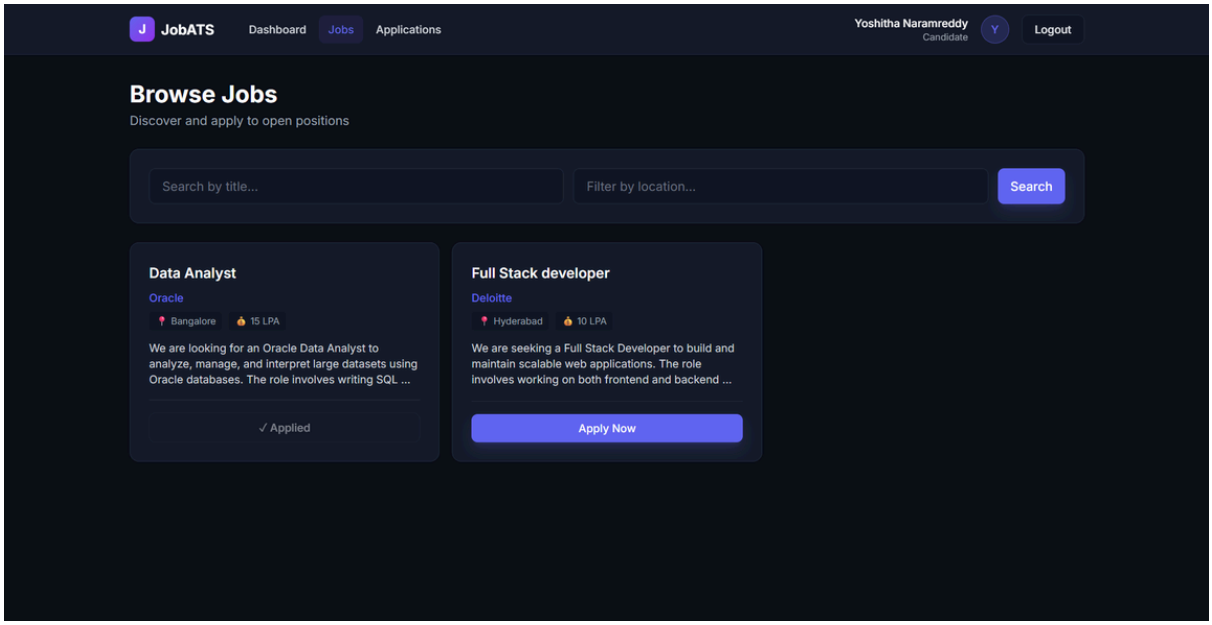
Candidate Dashboard:



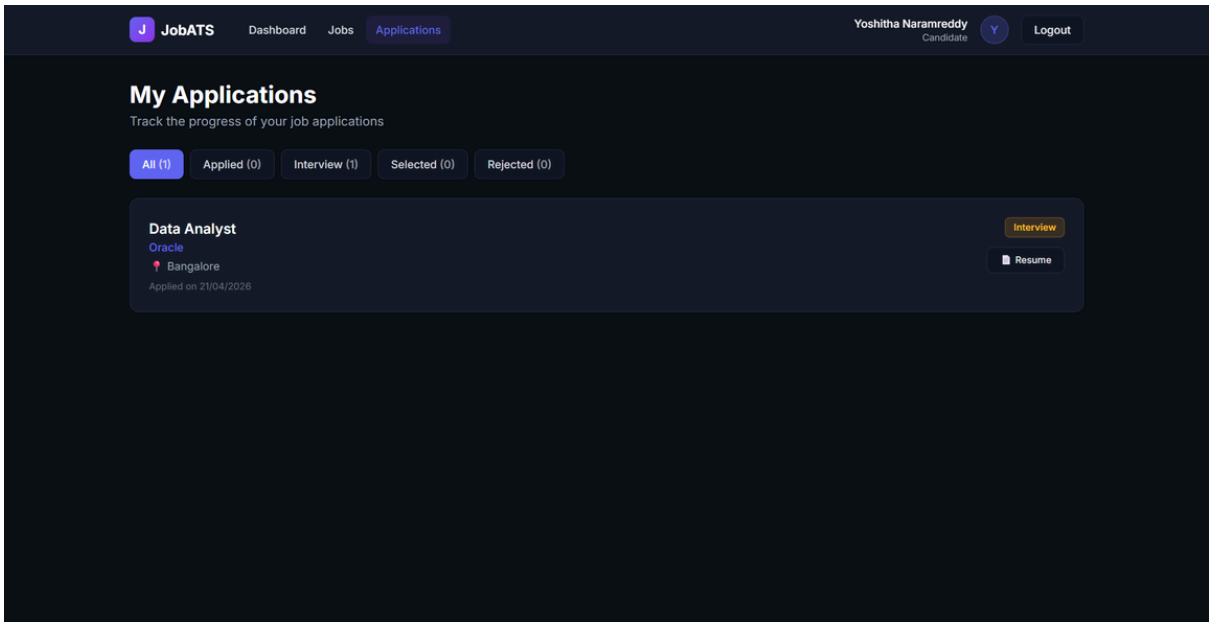
Recruiter Dashboard:



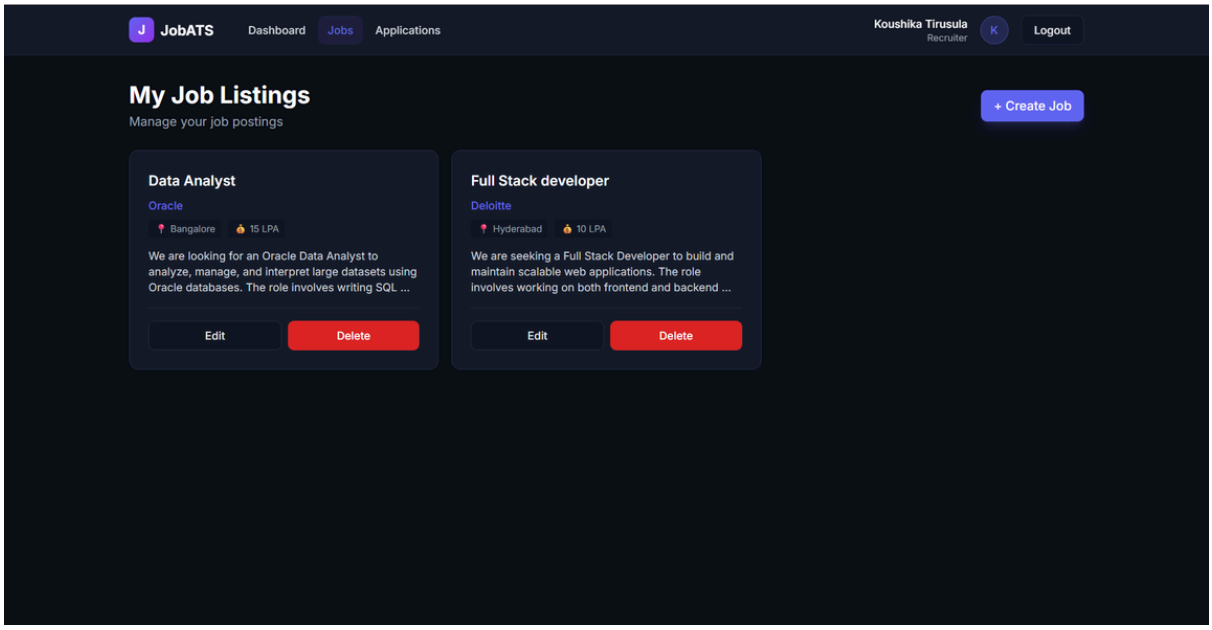
Browse Jobs Page:



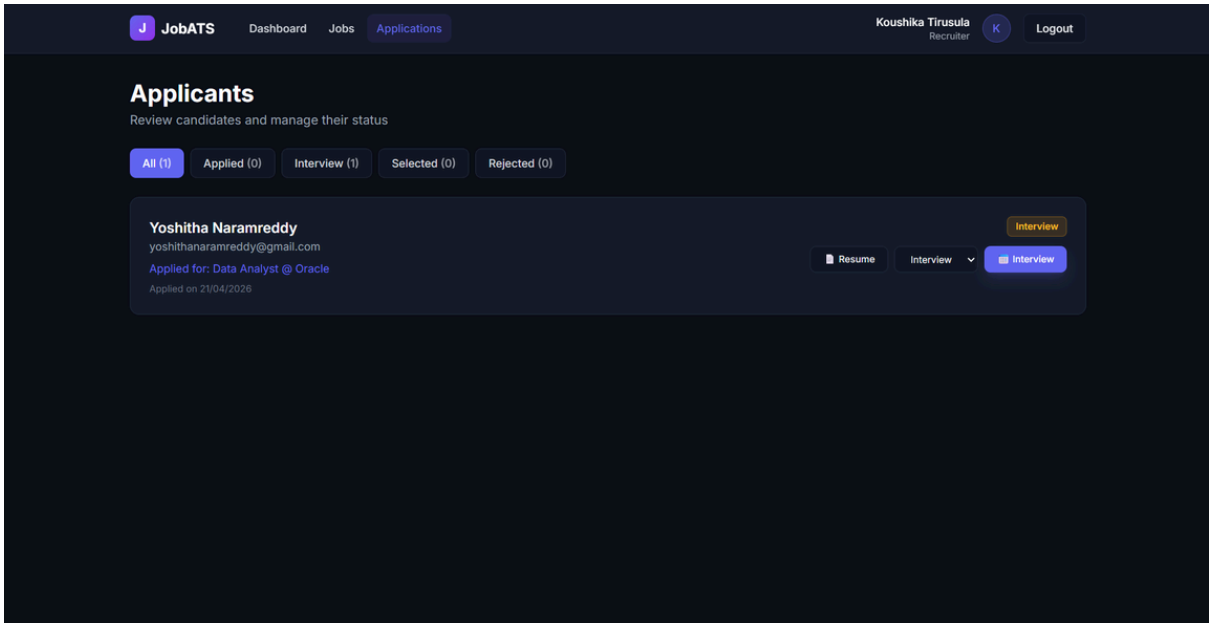
My Applications Page:



Job Listings Page:



Applicants Page:



Demo Video Link:

https://drive.google.com/file/d/1w0R48jiQ3FGponZQRZZiKONMa49zMs8J/view?usp=sharing

Code Drive Link:

https://drive.google.com/file/d/1HuI6n6M4Leu_EejA7nfwQ0MBJ6CgiPim/view?usp=sharing